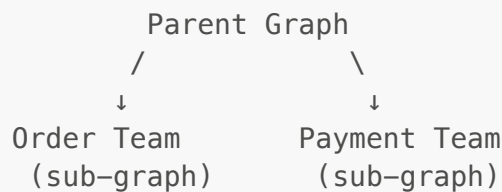


Hierarchical Agent Teams with LangGraph

As multi-agent applications grow, putting every agent into one large graph can make the system difficult to understand and maintain.

Hierarchical Agent Teams address this by grouping related agents into smaller **sub-graphs**.

For example, an order-processing system might contain:

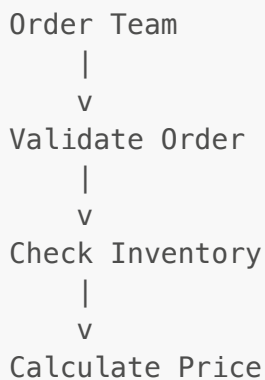


Each team manages its own internal workflow, while the parent graph coordinates the overall process.

1. Sub-Graphs

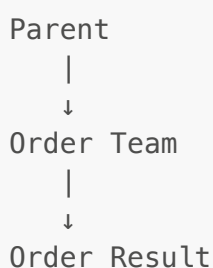
A sub-graph is a LangGraph graph that is used as a component inside another graph.

For example, the Order Team might internally contain:



The parent graph does not need to know about these internal steps.

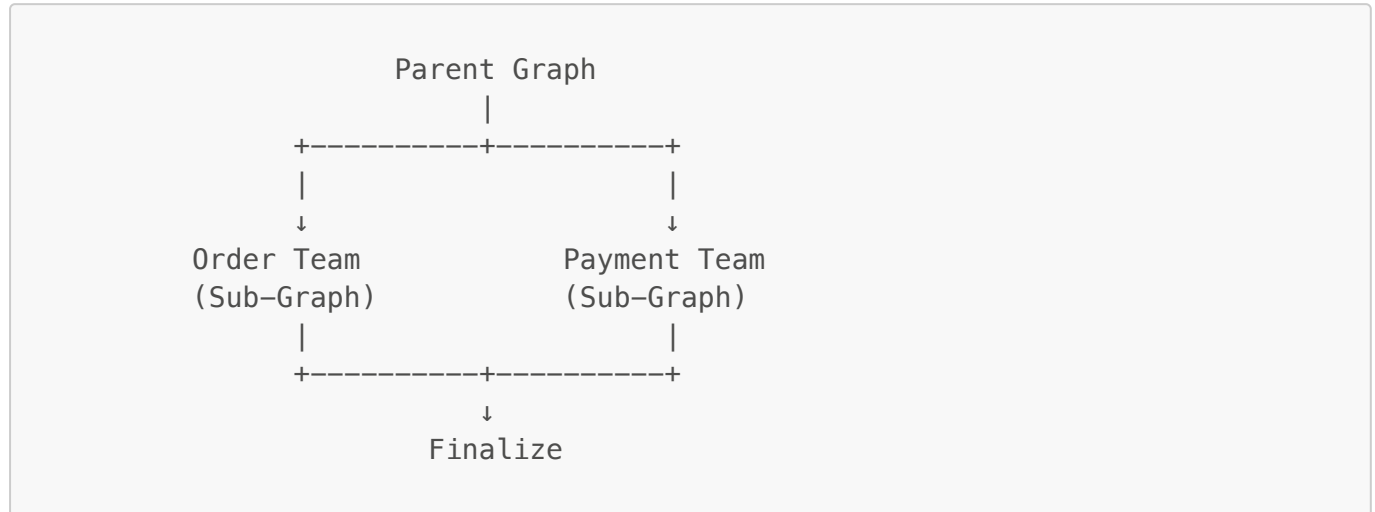
It simply interacts with the team:



This provides **encapsulation**.

2. Parent and Child Graphs

The overall architecture can look like:



The parent graph is responsible for the high-level workflow.

Each child graph is responsible for a specialized business capability.

This is similar to decomposing a large application into smaller modules.

3. Managing State

The parent graph can have a larger state:

```
class OrderState(TypedDict):  
    order_id: str  
    amount: float  
    order_result: str  
    payment_result: str
```

A child graph does not necessarily need all of this information.

The Order Team might only need:

```
{  
    "order_id": "ORD1001",  
    "amount": 25000  
}
```

The parent explicitly passes the required information to the child and receives its result.

```

Parent State
  |
  | order_id, amount
  ↓
Order Team
  |
  | validation result
  ↓
Parent State

```

Therefore, parent and child graphs can have **different state schemas**.

4. Why Use Separate State?

Separate state provides a useful boundary.

For example:

```

Parent State
├─ order information
├─ payment information
└─ final result

Order Team State
├─ order_id
└─ amount

Payment Team State
├─ order_id
└─ amount

```

The child only receives information relevant to its responsibility.

This can reduce unnecessary context and make individual teams easier to test.

5. Encapsulation

Without sub-graphs, a large application might become:

```

Main Graph
├─ Validate Order
├─ Check Inventory
├─ Check Payment
├─ Fraud Check
└─ Calculate Price

```

```
├─ Generate Invoice
└─ ...
```

With hierarchical teams:

```
Main Graph
├─ Order Team
│   ├── Validate
│   ├── Inventory
│   └─ Pricing
├─ Payment Team
│   ├── Payment
│   └─ Fraud
└─ Invoice Team
    └─ Generate Invoice
```

The hierarchy makes the overall architecture easier to reason about.

6. Latency Considerations

Hierarchical architecture introduces additional execution overhead.

Suppose the parent invokes three teams sequentially:

```
Parent
  ↓
Order Team      1 second
  ↓
Payment Team    2 seconds
  ↓
Invoice Team    1 second
```

The overall latency can approach the combined execution time.

If each team also makes LLM calls, the latency can increase further.

Therefore, not every function should become an agent or sub-graph.

Simple deterministic operations are often better implemented as normal Python functions.

7. Token Cost

LLM-based teams also consume tokens.

For example:

```
Parent LLM
  ↓
Order Team LLM
  ↓
Payment Team LLM
  ↓
Finalizer LLM
```

Each model invocation may consume input and output tokens.

A hierarchical design can therefore increase cost if every level independently invokes an LLM.

Good designs minimize unnecessary model calls and pass only the context required by each team.

8. When Hierarchical Teams Make Sense

Sub-graphs are particularly useful when:

- A capability contains several related agents or steps.
- A team has its own internal workflow.
- Different teams have different state requirements.
- The parent should not depend on internal implementation details.
- The system is large enough to benefit from modularity.

For a small workflow, a hierarchy may simply add unnecessary complexity.

Key Takeaway

Hierarchical agent architecture introduces another level of abstraction:

```
Parent Graph
  ↓
Sub-Graph / Team
  ↓
Specialized Agents
  ↓
Result
```

The **parent graph coordinates**, while each **sub-graph encapsulates a specialized capability**.

The main benefits are modularity, encapsulation, and clearer state boundaries. The main architectural costs are additional **latency, LLM calls, and token consumption**.

The goal is not to maximize the number of agents or graphs, but to use hierarchy where it provides meaningful separation of responsibilities.